

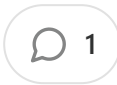
El stack de Gentle AI: ¿Como se desarrolla software en 2026?

El antes y despues en la manera de escribir codigo



NICOLAS LOPEZ

JUN 01, 2026



Compartir

Hace unos meses cuando arrancaba una sesión con un agente para codear era empezar siempre de cero, le explicaba lo mismo tres veces, se olvidaba decisiones que ya habíamos tomado, y terminaba tocándome archivos que no tenían nada que ver con lo que le pedí o cargandome el context windows ya empezando con un monton de % cargado.

Probé un montón de cosas para arreglar eso:

- prompts más largos
- instrucciones más detalladas
- system prompts cargados de reglas

Nada me cambiaba el resultado a nivel estructural/resultado.

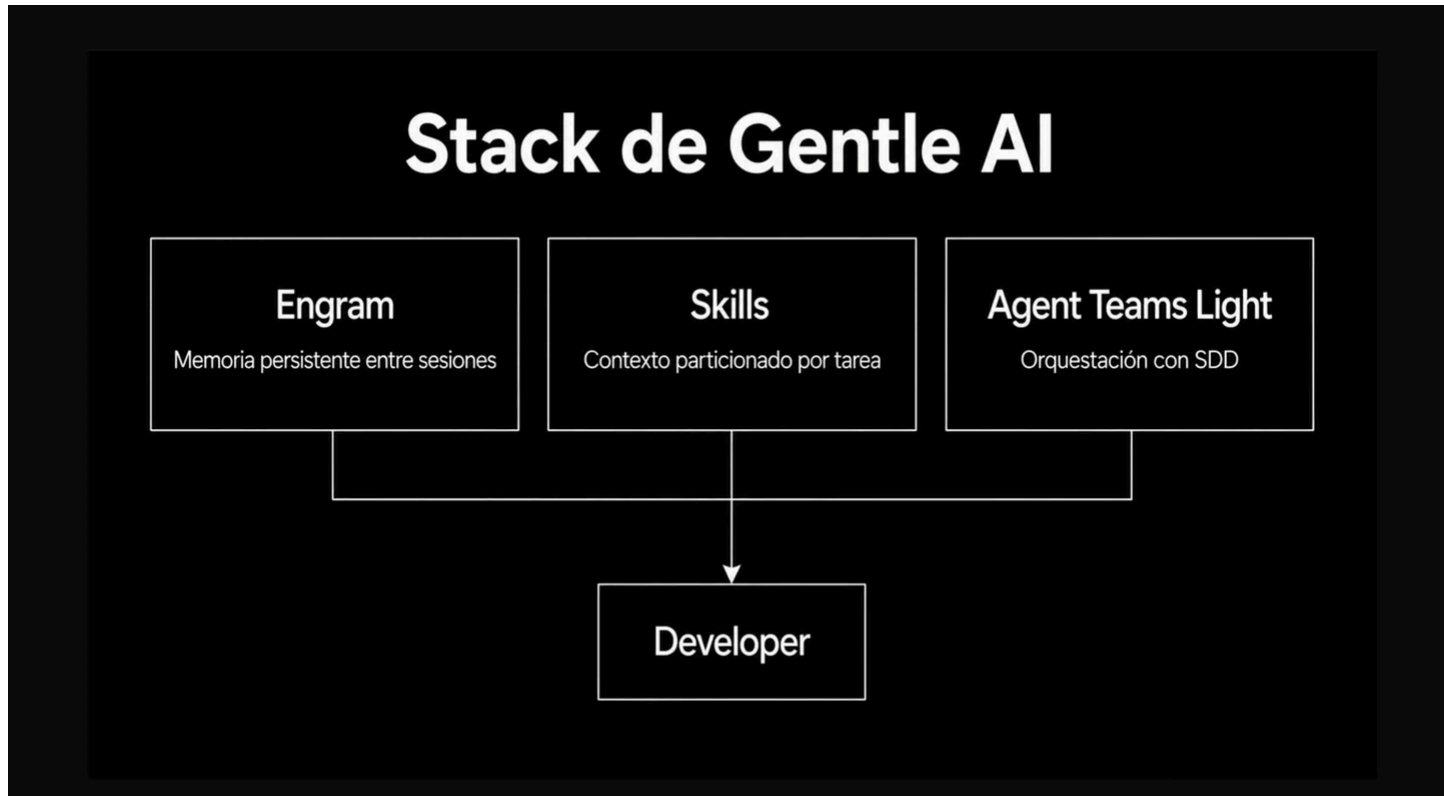
Hasta que arranqué a usar el stack de Gentle AI, que es lo que viene armando

[@G Programming](#)

como ecosistema para laburar con agentes

El stack tiene **tres piezas** que se integran entre sí y resuelven, cada una, un problema distinto

- **Engram** le da memoria que persiste entre sesiones,
- **Skills** le particionan el contexto para que no cargue todo de una
- **Agent Teams Light** es el orquestador que coordina el flujo de trabajo entero usando **SDD**.



Todo se instala con un solo comando

```
brew install gentleman-programming/tap/gentleman-ai
```

y de ahí en adelante tenés el binario de Engram listo para correr, el orquestador instalado, los skills predefinidos cargados y todo pipícucú

Engram, la memoria que sobrevive entre sesiones

Engram es básicamente la **memoria del agente**

Es lo que le permite acordarse de decisiones, bugs, convenciones y preferencias que tomaste en sesiones anteriores, así no le tenés que explicar lo mismo cada vez que abrís el chat

Funciona como un servicio chiquito que corre en tu máquina, sin nube, sin APIs externas. Por debajo guarda todo en un archivo local en tu computadora. Eso significa que la lectura es instantánea, no pagás por query, y **nada de lo que guardás sale de tu máquina**

Lo que el agente guarda se llama **observación**, cada **observación** tiene:

Observación de Engram

title	Implementé retry con exponential backoff
type	decision
topic_key	http-client/retry-policy
content	• qué se hizo • por qué • qué se aprendió

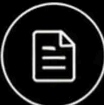


- Un título corto y buscable
- Un tipo (bug arreglado, decisión arquitectónica, convención del equipo, algo descubierto del codebase, etc.)
- El nombre del proyecto al que pertenece
- Un cuerpo que explica qué se hizo, por qué, en qué archivos, y qué se aprendió en el camino

Hay un detalle del diseño que me parece importante y es el **topic key**.

Es una **etiqueta estable** que identifica el tema de la **observación**. Si una convención cambia o una decisión se revisa, el agente usa el mismo **topic key** y Engram **actualiza la observación existente** en lugar de crear una nueva. Sin eso, terminás con cinco versiones contradictorias de la misma decisión y el agente arranca a tomar decisiones con info incoherente.

El agente no tiene que hacer nada raro para usar Engram, simplemente llama herramientas con nombres descriptivos. Las que más vas a ver son:

 Herramientas de Engram	
 guarda o actualiza una observación	<code>mem_save</code>
 busca observaciones por texto	<code>mem_search</code>
 trae el contenido completo de una	<code>mem_get_observation</code>
 trae el historial reciente del proyecto	<code>mem_context</code>
 cierra la sesión con un resumen	<code>mem_session_summary</code>

Vos lo vas a ver así:

```
⊗ engram_mem_search [limit=5, project=ecommerce-argentina, quer  
apply-progress, scope=project, type=]  
  
⊗ engram_mem_search [limit=5, project=ecommerce-argentina, quer  
ecommerce-argentina, scope=project, type=]  
  
# Checks status before apply in projects/ecommerce-argentina  
  
$ git status --short --branch  
  
## master...origin/master
```

Guardado proactivo, la regla que a mí personalmente me encanta

Engram solo funciona si el agente guarda proactivamente, no solo cuando se lo pedís



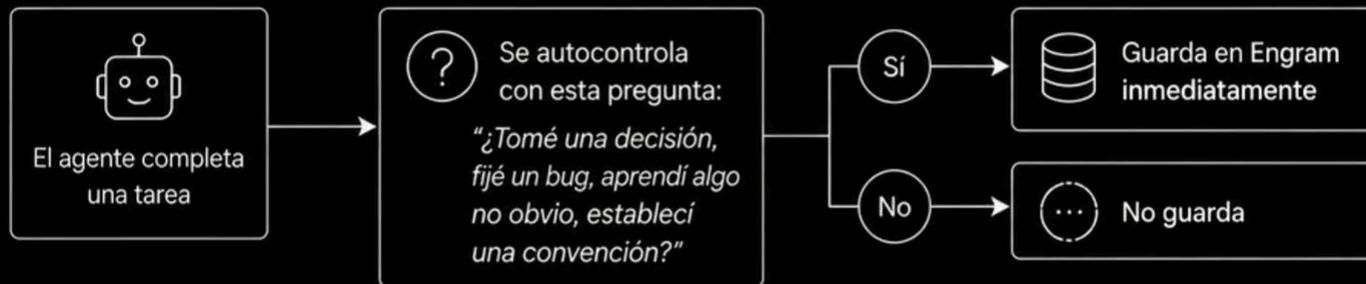
Guardado proactivo en Engram

Engram solo funciona si el agente guarda proactivamente, no solo cuando se lo pedís.



Política simple:

El agente hace el save **inmediato y sin pedir permiso** después de cada:



¿Qué dispara el guardado automático?



Decisión de arquitectura tomada



Bug fijado
(incluyendo la causa raíz)



Convención o patrón establecido



Comportamiento no obvio del codebase descubierto



Preferencia o restricción tuya aprendida



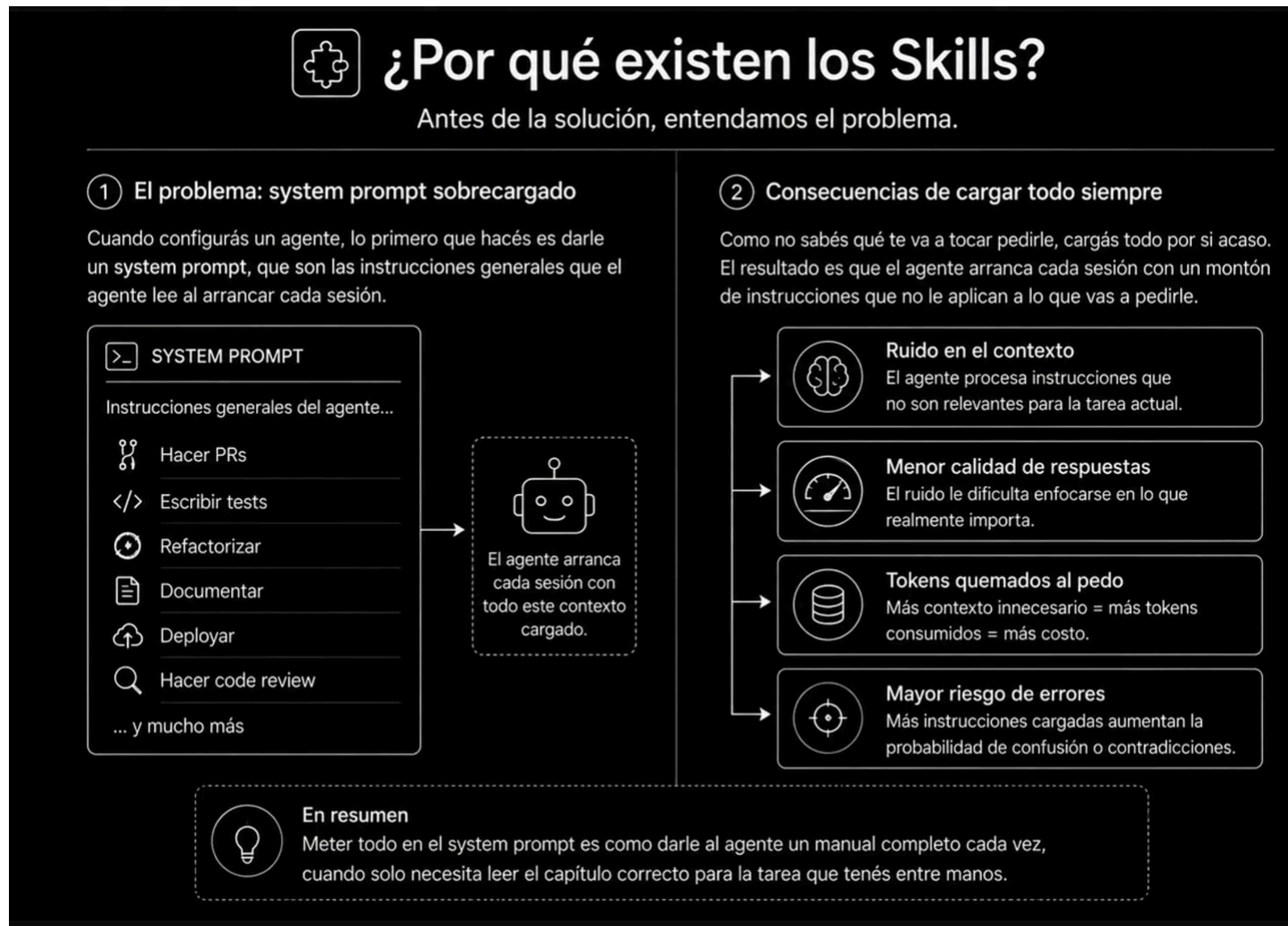
Esto permite que, al volver al proyecto, el agente sepa exactamente en qué andabas, qué decisiones se tomaron y qué aprendió en el camino.

El agente se autocontrola con una pregunta simple después de cada tarea: "¿tomé una decisión, fijé un bug, aprendí algo no obvio, establecí una convención?". Si la respuesta es sí, es guardado al toque

Eso es lo que después te permite abrir el proyecto mañana y que el agente sepa exactamente en qué andabas

Skills, instrucciones que se cargan solo cuando hacen falta

¿Para que sirven y existen los Skills?



Como no sabés qué te va a tocar pedirle, cargás todo por si acaso. El resultado es que el agente arranca cada sesión con un montón de instrucciones que no le aplican a lo que vas a pedirle, y **ese ruido le baja la calidad de las respuestas además de quemarte tokens al pedo.**

Los skills resuelven esto cargando las instrucciones solo cuando hacen falta. En vez de tener todo el manual del agente abierto desde el primer minuto, cada tarea tiene su propio capítulo en un archivo separado, y el agente lo va a buscar cuando detecta que lo necesita.

Cada skill es un archivo de texto con tres partes simples.

1. **El trigger:** define cuándo se carga (por ejemplo, “cuando el usuario pide hacer un PR” o “cuando se va a tocar un archivo de tests”)
2. **El contexto:** son las instrucciones específicas para esa tarea, con ejemplos, restricciones y convenciones
3. **El output contract:** define qué tiene que entregar el agente al terminar

Anatomía de un Skill



El trigger es lo más importante porque si está mal definido el skill nunca se carga o se carga cuando no toca, y todo el sistema deja de funcionar como debería.

El skill registry, que es básicamente el índice

Para que el agente sepa qué skills hay disponibles sin tener que cargarlos todos, **existe un archivo central que funciona como índice.**

Es un Markdown que lista cada skill con su nombre, una descripción corta del trigger, y la ruta exacta al archivo del skill.

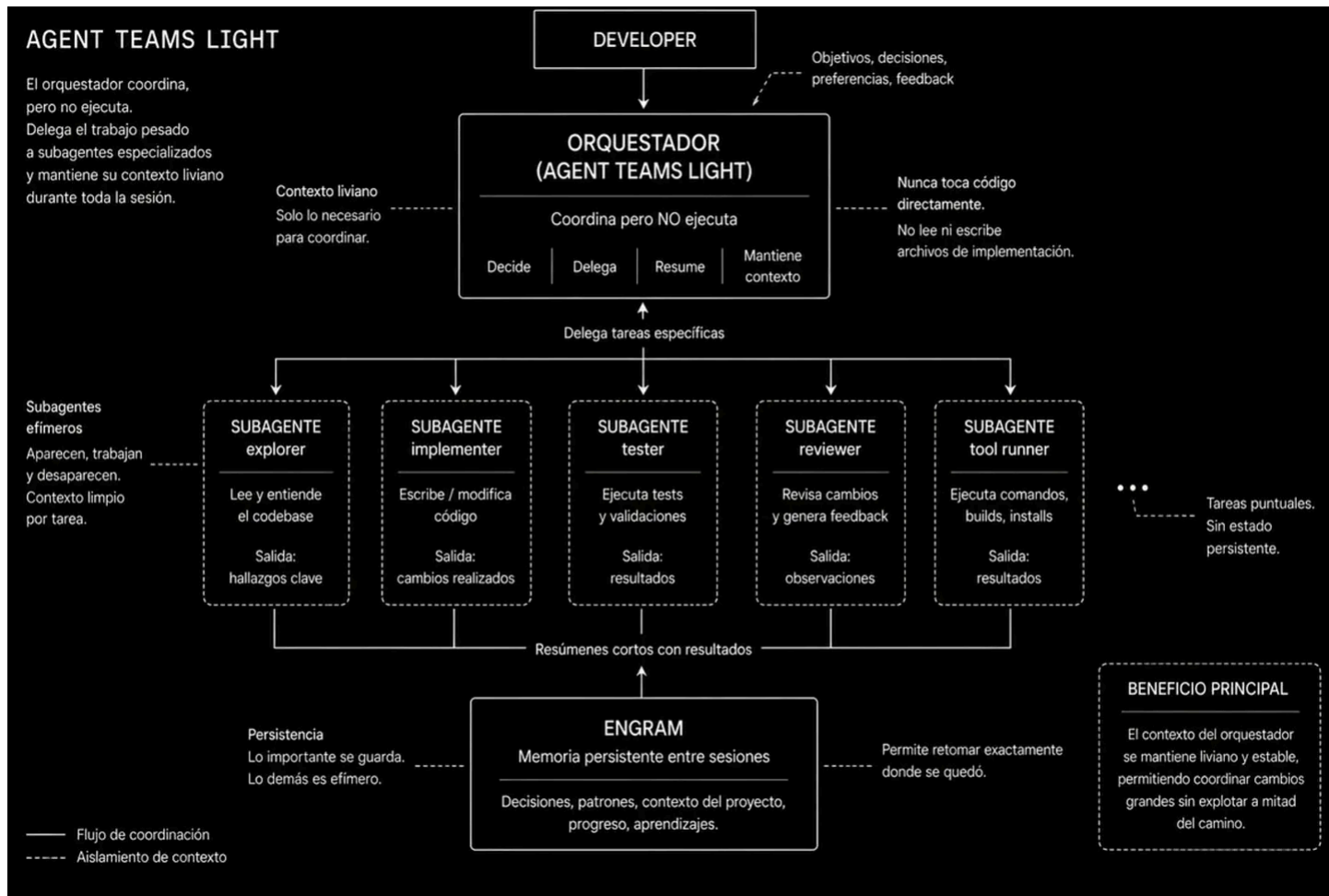
Vive en dos lugares posibles según para qué lo uses. Hay uno por proyecto que va junto al código y es específico de ese repo (cosas tipo “para nuestros PRs usamos este formato”). Y hay uno global del usuario, que vive en tu carpeta personal y aplica a todos tus proyectos (cosas tipo “siempre hacé commits con conventional commits”).

Cuando el agente arranca o cuando pierde el contexto, lee este índice primero. Solo el índice, no todos los skills. Después, cada vez que delega una tarea a un subagente, hace match de qué skills aplican a esa tarea específica y le pasa al subagente las rutas exactas para que las lea antes de empezar.

Y acá hay un detalle de diseño que parece menor pero es importante: o que se pasan rutas a los archivos, no el contenido de los archivos. El subagente lee los archivos

originales por su cuenta. Eso parece más trabajo pero te protege de un problema típico, si el orquestador resume mal o se le mezcla algo del contexto, el subagente igual lee el skill original y la intención del que lo escribió queda intacta.

Agent Teams Light, el orquestador SDD



Agent Teams Light es la pieza que conecta todo y le da estructura a cómo el agente trabaja. Pensalo como una persona que no toca el código directamente, sino que coordina a otros agentes especializados para que cada uno haga la parte que le toca. Algo así como tu PM que solo te dice "como venimos chicos?" y se vuelve a dormir la siesta (chiste)

La regla principal del orquestador es que coordina pero no ejecuta. Esto que parece sutil cambia todo, porque su contexto tiene que mantenerse liviano (es el hilo que persiste durante toda la sesión).

Si el orquestador se pone a leer 20 archivos y a escribir código, su contexto se llena de detalles y deja de poder coordinar bien. Por eso delega casi todo el trabajo pesado a subagentes que arrancan limpios, hacen una tarea específica, reportan, y se cierran.

La división de qué hace y qué delega es bastante clara:

- El orquestador hace inline cuando alcanza con leer uno a tres archivos para decidir, cuando tiene que escribir un archivo mecánico que ya conoce, o cuando solo necesita correr comandos de git para chequear estado.
- Y delega a un subagente cuando necesita leer cuatro o más archivos para entender algo, cuando va a escribir con análisis en múltiples archivos, o cuando tiene que ejecutar tests, builds o instalaciones.

Hay también reglas que disparan delegación obligatoria:

⚠ Reglas que disparan delegación obligatoria

Estas reglas protegen el contexto del orquestador. Cuando se cumple cualquiera de ellas, el orquestador debe delegar la tarea a un subagente especializado.

1 Regla de cuatro archivos



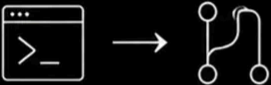
Si entender algo requiere abrir cuatro o más archivos, delegar una exploración.

2 Regla de multi-file write



Si vas a tocar dos o más archivos no triviales, delegar la implementación.

3 Regla de PR



Antes de commit, push o PR, hacer un review fresco delegando.

4 Regla de sesión larga



Después de ~20 tool calls sin delegar, pausar y delegar.

→ | Esto que parece detalle es lo que evita que el contexto del orquestador explote a mitad de un cambio grande.

Los comandos SDD

Acá hay **dos tipos de comandos** según cómo el agente los procesa. Los que aparecen en el autocompletado de Claude Code (O de opencode/lo que uses) cuando tipeás

/sdd:

```

COLFARMA_IMPORT_PASSWORD

Y luego correr:

/sdd-new      Start a new SDD change - runs exploration then creates a proposal
/sdd-apply    Implement SDD tasks - writes code following specs and design
/sdd-onboard  Guided SDD walkthrough - onboard a user through the full SDD cycle using their real codebase
/sdd-ff       Fast-forward all SDD planning phases - proposal through tasks
/sdd-continue Continue the next SDD phase in the dependency chain
/sdd-init     Initialize SDD context - detects project stack and bootstraps persistence backend
/sdd-archive  Archive a completed SDD change - syncs specs and closes the cycle
/review       review changes [commit|branch|pr], defaults to uncommitted
/sdd-explore  Explore and investigate an idea or feature - reads codebase and compares approaches
/sdd-verify   Validate implementation matches specs, design, and tasks

/ssd

Sdd-Orchestrator-Nicolopez · GPT-5.5 OpenAI
27.8K (7%)  ctrl+p  commands

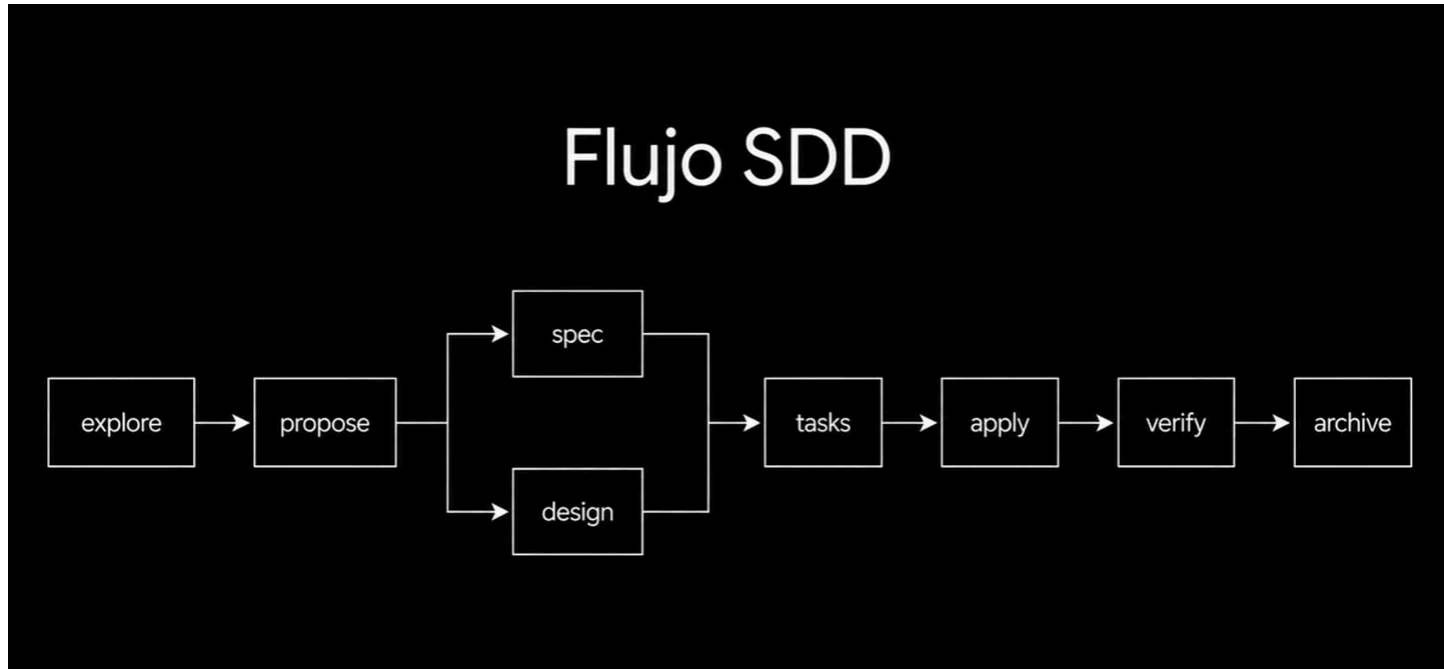
```

- **sdd-init:** inicializa el contexto del proyecto, detecta stack y testing
- **sdd-explore:** investiga una idea, lee el codebase, no crea artefactos
- **sdd-apply:** implementa las tareas en batches, marca completadas
- **sdd-verify:** valida la implementación contra specs, reporta los hallazgos como críticos, warnings o sugerencias
- **sdd-archive:** cierra el cambio y persiste el estado final

Y después hay meta-commands que el orquestador maneja por debajo sin autocompletado. **El sdd-new** arranca un cambio nuevo (delega la exploración y la propuesta), **el sdd-continue** ejecuta la fase que sigue, y **el sdd-ff** es un fast-forward que te corre proposal, specs, design y tasks en secuencia.

El flujo completo de un cambio

Cada cambio pasa por varias fases que se encadenan en este orden.



- **Explore**, que es investigación pura sin tocar nada.
- **Propose**, que es la idea formal de qué se va a hacer.

De ahí se abre en dos ramas que corren en paralelo:

- **Spec** describe qué tiene que pasar en términos de comportamiento

- **Design** es cómo se va a lograr técnicamente.

Ambas dependen de la propuesta aprobada. Las dos confluyen en:

- **Tasks**, que es la descomposición en pasos chicos.

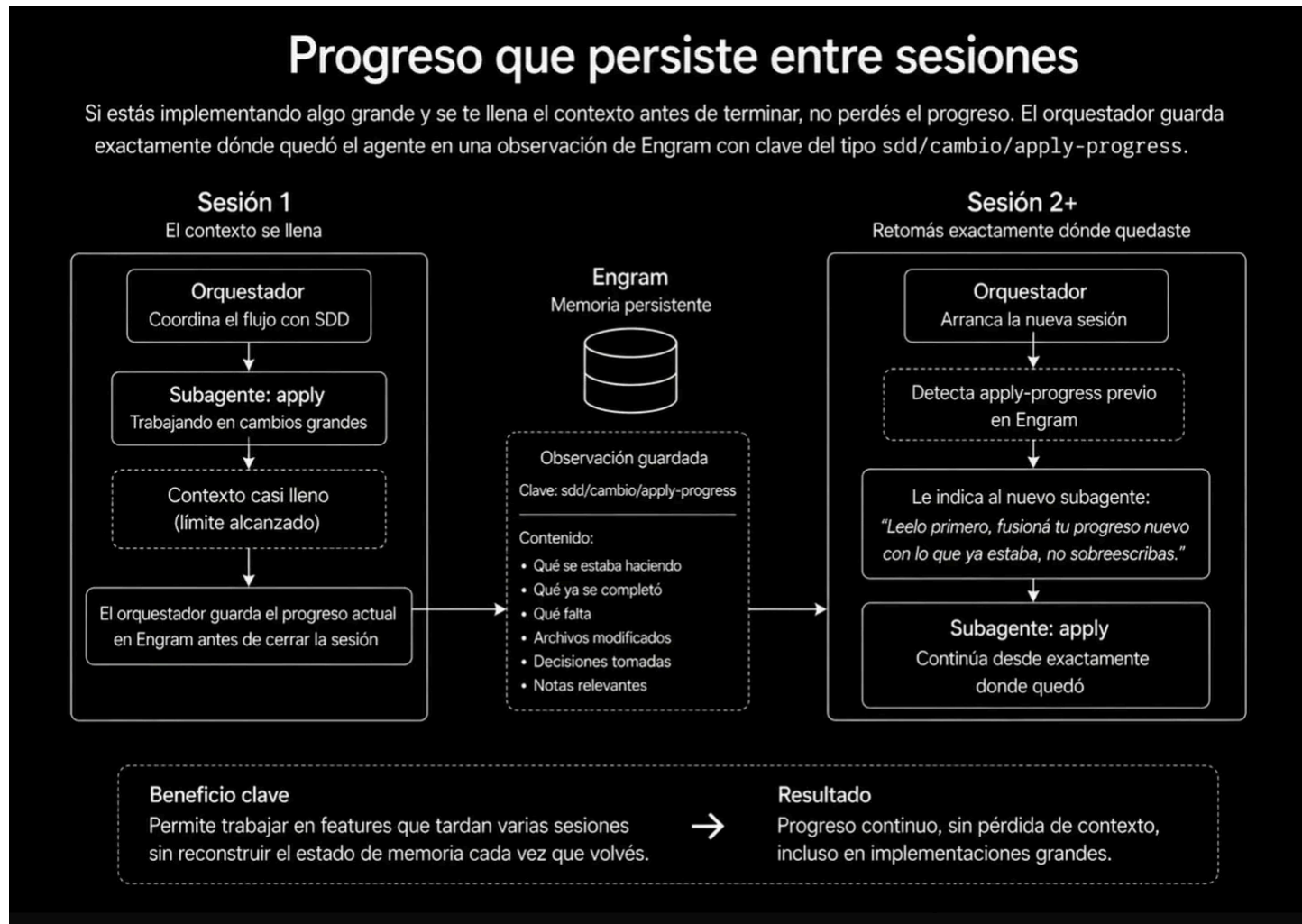
Después viene:

- **Apply** que es la implementación
- **Verify** que es la verificación contra spec
- **Archive** que cierra y guarda todo

Spec y Design corren en paralelo porque son independientes entre sí, las dos dependen de que la propuesta esté aprobada, y Tasks depende de ambas para arrancar. Esto te acorta el tiempo total del ciclo sin pisar dependencias.

Apply-Progress, para cuando un cambio no entra en una sola sesión

Si estás implementando algo grande y se te llena el contexto antes de terminar, no perdés el progreso. El orquestador guarda exactamente dónde quedó el agente en una observación de Engram con clave del tipo `sdd/cambio/apply-progress`.



Cuando volvés a la sesión, el orquestador detecta que hay un apply-progress previo y le dice al nuevo subagente "leelo primero, fusioná tu progreso nuevo con lo que ya estaba, no sobrescribas".

Eso te permite trabajar en features que tardan varias sesiones sin tener que reconstruir el estado de memoria cada vez que volvés, que es justo lo que pasa cuando armás algo grande con un agente común.

El forecast del review

Después de que `sdd-tasks` termina de descomponer el cambio, el sistema te estima qué tan grande va a ser el PR. Si pasa de 400 líneas o detecta que conviene dividirlo, te avisa y aplica la estrategia que cacheaste al principio:

- **Ask on risk:** te pregunta si dividir en PRs encadenados o seguir con una excepción explícita de tamaño.
- **Auto chain:** no pregunta, implementa en pedazos autónomos con boundaries claros.
- **Single PR:** requiere que vos marques una excepción explícita antes de implementar.
- **Exception ok:** continúa pero te avisa que el run usa excepción de tamaño.

Si vas con PRs encadenados, elegís entre dos estrategias:

1. **stacked to main:** donde cada PR mergea a main en orden y te permite iteración rápida.

2. **feature branch chain:** donde todo va apilado en una branch tracker y solo el tracker mergea a main, que es mejor para rollback.

Esto que parece un detalle de entrega es lo que evita que mandes un PR de 1000 líneas que **nadie quiere revisar**

Cada fase usa el modelo que mejor le calza

Una de las cosas más copadas del diseño es que cada fase no usa el mismo modelo, sino el que tiene sentido para esa tarea específica. Lo pongo de ejemplo con los modelos de Claude:

Cada fase usa el modelo que mejor le calza

Fase	Modelo
explore	Sonnet
propose	Opus
spec	Sonnet
design	Opus
tasks	Sonnet
apply	Sonnet
verify	Sonnet
archive	Haiku

- **Explore - Sonnet:** porque es lectura y comprensión estructural.
- **Propose - Opus:** porque son decisiones arquitectónicas.
- **Spec - Sonnet:** porque es escritura estructurada.

- **Design - Opus:** porque son decisiones técnicas con consecuencias largas.
- **Tasks - Sonnet:** porque es descomposición mecánica.
- **Apply - Sonnet:** porque es implementación concreta.
- **Verify - Sonnet:** porque es validación contra spec.
- **Archive - Haiku:** porque es cierre y persistencia, sin razonamiento pesado.

La calidad no baja porque cada fase realmente necesita un perfil distinto de modelo.

Strict TDD Mode

Acá hay una de las decisiones de diseño que más me cierra de todo el stack, y te lo digo desde mi background de QA =P

Si el proyecto tiene capacidades de testing, el `sdd-init` las detecta y activa lo que se llama **Strict TDD Mode**. A partir de ahí, cada vez que el orquestador lanza un `apply` o un `verify`, le inyecta al subagente la regla de que tiene que seguir el flujo TDD clásico de “primero el test, después el código”

El flujo es el conocido **red-green-refactor** pero con un agregado importante, triangulación.

El agente convierte los criterios de aceptación de la spec en test cases, los corre y deben fallar (red), escribe el mínimo código para que pasen (green), refactoriza sin romper los tests (refactor), y después busca dos posibilidades adicionales que podrían romper el código y las testea también.

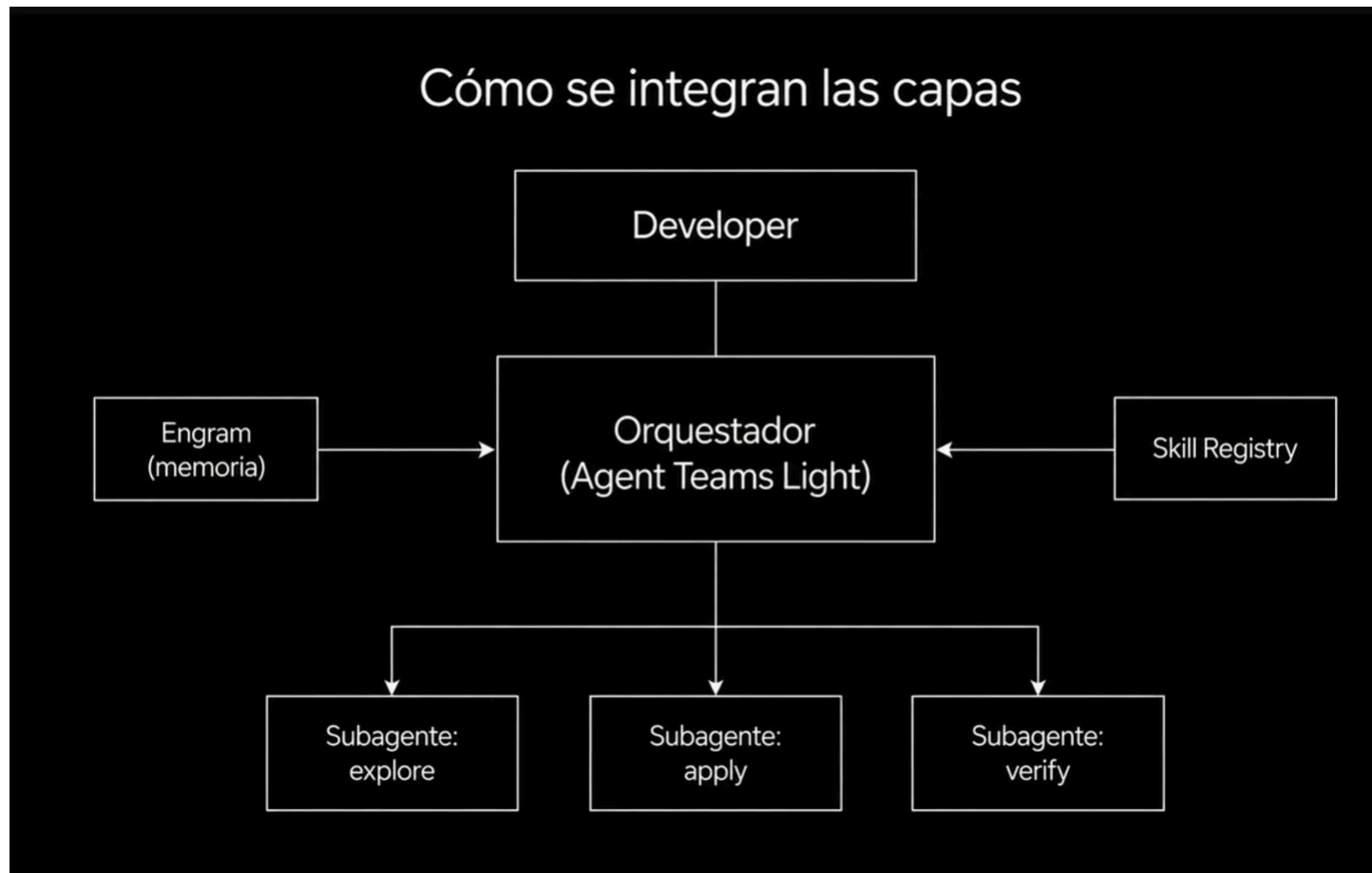
Y acá viene la parte que me parece subestimada y vale la pena marcar:

TDD con un agente no es lo mismo que TDD con un humano. Cuando un dev programa sin tests, vos podés ir al código, leer la lógica, y entender qué hizo, el razonamiento queda inscripto en el código mismo, cuando un agente programa sin tests, esa garantía no existe.

El agente puede haber tomado un atajo, haber asumido algo que la spec no decía, o haber implementado solo el happy path porque era el camino más corto.

Entonces TDD con agentes deja de ser “buena práctica” y **pasa a ser el único contrato verificable entre vos y el modelo**. Es lo que separa “el agente dice que terminó” de “el agente terminó”.

Cómo se integran las capas



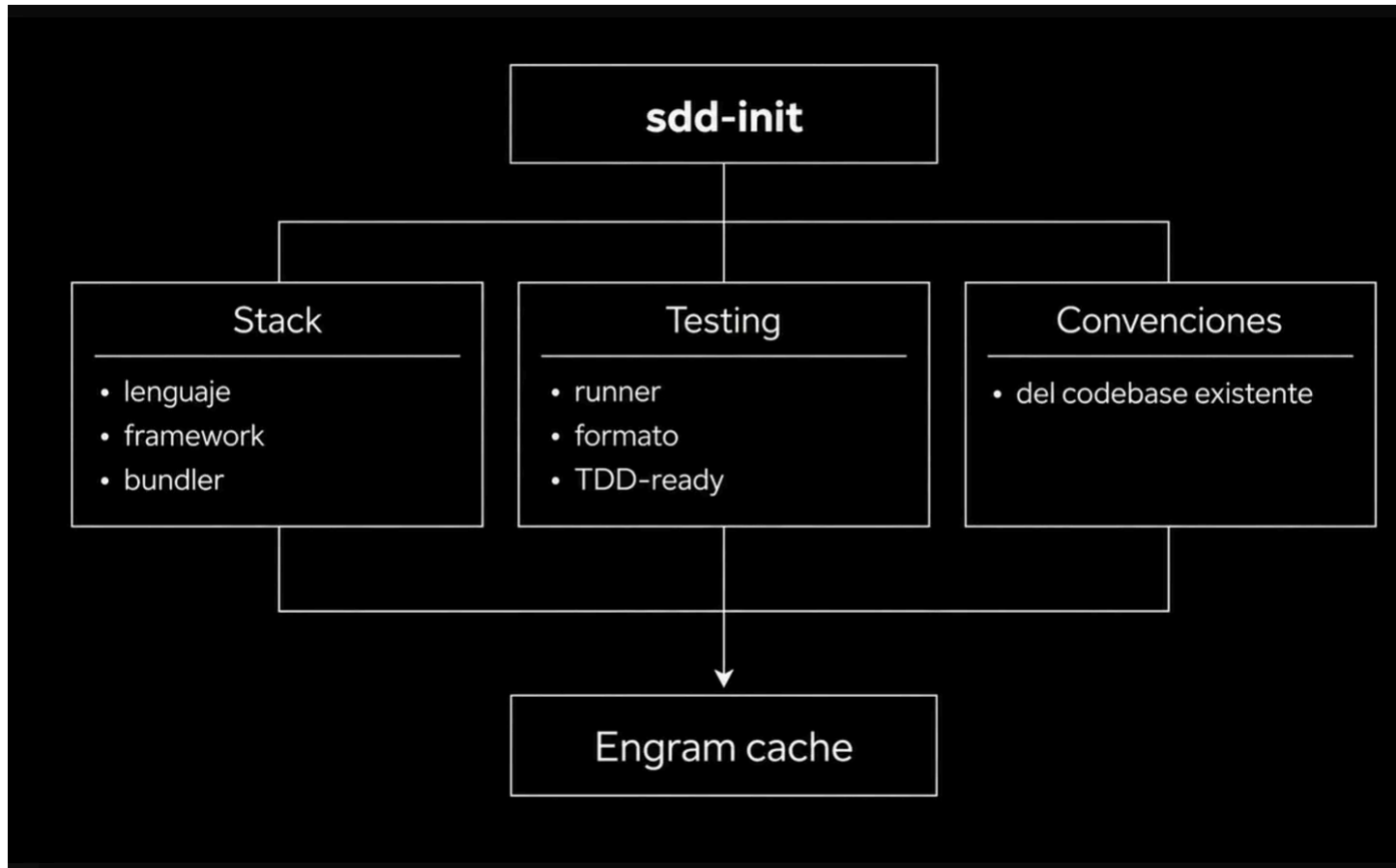
El dev ve un solo hilo de conversación, por debajo, el orquestador está lanzando y coordinando subagentes con contextos independientes. El usuario manda un mensaje, el orquestador consulta el skill registry para cargar skills relevantes, consulta Engram para recuperar contexto del proyecto, lanza subagentes con el contexto mínimo que cada uno necesita, y va guardando el progreso en Engram a medida que avanza.

Cada subagente arranca limpio con un briefing específico, hace su trabajo, reporta, y se cierra.

Solo el resumen sube al orquestador, el contexto del orquestador crece linealmente porque acumula resúmenes, no implementaciones, y el de cada subagente es acotado porque solo recibe lo mínimo para su tarea.

Eso es lo que te permite trabajar en cambios grandes sin que el contexto explote.

La guardia de `sdd-init`



Antes de ejecutar cualquier comando SDD, el orquestador chequea si el init fue corrido para el proyecto buscando una observación con clave `sdd-init/proyecto` en Engram. Si la encuentra, sigue normal. Si no la encuentra, corre el init en silencio primero y después continúa con lo que pediste.

El init detecta varias cosas en un solo paso:

- Stack del proyecto (lenguaje, framework, bundler)
- Capacidades de testing (runner, formato de tests, si es TDD-ready)
- Convenciones existentes en el codebase
- Si corresponde, activa Strict TDD Mode

Toda esa info queda cacheada en Engram y disponible para todas las fases del ciclo SDD sin tener que re-detectar.

Una nota importante sobre la configuración

Hay un punto que vale la pena marcar:

Tu configuración tiene que ser agnostica

Los skills y el contexto inicial son estándares y se pueden reusar entre proyectos y entre agentes.

Eso significa que cuando armás un setup que funciona, no estás atado a un solo modelo o un solo IDE. Si mañana querés cambiar de Claude Code a otro, los skills se reusan, la memoria se reusa, el flujo se reusa.

Esa es la parte que más me cierra del diseño de Alan, que no está construyendo un producto cerrado sino un ecosistema composable donde cada pieza tiene un rol claro

y se puede usar por separado o todas juntas.

Si te interesa este tipo de tema técnico, en el próximo me meto con Engram Cloud, SDD Judge day y demás, que son una bomba.

Suscríbete a Nicolas Lopez

Launched 2 months ago

Testing, IA y carrera profesional: cómo adaptarte sin quedarte atrás.

Escribe tu correo electróni

Suscribirse

Al suscribirte, aceptas los [Términos de uso](#) de Substack y reconoces su [Aviso de recopilación de información](#) y su [Política de privacidad](#).

Discusión sobre este post

Comentarios

Restacks



Escribe un comentario...



Cristian Moreno 11h

...

Está genial el artículo me gustó mucho. Sería interesante compartir la forma de trabajar en diferentes momentos, como por ejemplo: casando un bug, creando un nuevo feature, agregándole nuevos detalles al feature inicial, eliminando un feature completo y que engran entienda, entre otros detalles



ME GUSTA



RESPONDER



COMPARTIR

© 2026 Nicolas Lopez · [Privacidad](#) · [Términos](#) · [Aviso de recolección](#)
[Substack](#) es el hogar de la gran cultura